

Project Plan:

AI-Assisted Cloud Kitchen Inventory Simulation

1. Why AI-Assisted Coding for This Case

- Speeds up first implementation from data model to runnable logic.
- Helps quickly refactor when business rules change.
- Makes it easy to test “what if” logic through prompt iterations.
- Supports clear separation between logic changes and output-format changes.

2. Business Context

Cloud kitchens operate on tight margins. After aggregator commissions (typically 20–30% of order value), packaging costs, marketing spend, food costs, rent, labor, and utilities, contribution margins on a typical order are often only 5–15%. Even small operational inefficiencies have a meaningful impact on profitability.

Why This Problem Matters Financially

- **Stockouts:** Stockouts cause lost sales: every minute an ingredient is unavailable but its dependent menu items remain listed leads to order acceptances the kitchen cannot fulfill, hurting both revenue and platform ranking.
- **Food waste:** Food waste hurts margin: in poorly managed kitchens, 8–15% of food cost is thrown away each month due to expiry. A recipe-linked inventory system can cut this significantly.
- **Customer experience:** Customer experience drives ranking: rejected orders, missing items, and partial deliveries hurt aggregator ratings, reducing visibility and orders — a downward spiral.

3. Problem Statement

A cloud kitchen is a delivery-only food business where multiple virtual brands operate from the same kitchen, sharing staff, equipment, and ingredients. While this improves utilization and revenue, it creates major operational challenges in inventory and recipe management.

Key Problems Being Solved

- **Silent stockouts:** Ingredients run out mid-shift, causing rejected orders.
- **Manual menu updates:** When an ingredient is unavailable, dependent items across brands and platforms must be manually disabled.
- **No expiry tracking:** Lack of freshness monitoring leads to food waste and risk of expired ingredients being used.
- **Inefficient restocking:** Poor consumption visibility causes over-ordering or under-ordering.
- **No structured procurement planning:** Kitchens lack a prioritized, data-driven daily restock plan.

4. Mental Model: The Director, Not the Typist

Before writing a single prompt, learners must internalize the correct mental model for AI-assisted coding.

AI is not a software engineer. It is a probabilistic generator — an advanced autocomplete system trained on billions of lines of code. It predicts what looks plausible, not what is correct.

Think of it as a confident but forgetful junior developer: writes fast, has seen a lot of examples, sometimes gets details wrong, and forgets what you told it 10 minutes ago.

Three Roles You Play Throughout This Project

- **Navigator:** Set the direction and goal. Break work into clear, small steps. Keep the big picture in focus.
- **Reviewer:** Inspect every AI-generated output. Ask: Does this do what I asked? Spot gaps, errors, or drift from intent.
- **Decision-Maker:** Accept, reject, or redirect AI output. Own the final quality of the work. You are always accountable.

5. Solutioning Approach: AI-Assisted Coding

We are building a Python-based simulation of a multi-brand cloud kitchen's inventory and order management for a single operational day. The system processes orders in real time, tracks shared inventory and ingredient expiry, supports partial fulfillment, and performs periodic restocking.

Core Idea

Each menu item is linked to a recipe. When an order arrives, the system:

1. Looks up the recipe for each item and calculates the total ingredients required
2. Checks inventory for sufficient stock and valid expiry dates
3. Deducts the used ingredients and marks the order as Delivered if all stock is available
4. Flags the order as Not Delivered and adds missing or expiring items to the restock list if stock is insufficient

Restocking happens at configurable checkpoints (every N orders), not after every order.

Key Design Principles

- **Centralized inventory:** All brands consume from one shared inventory source.
- **Recipe-driven deduction:** Inventory updates happen automatically from recipes and order quantities.
- **Availability validation:** Ingredients must have sufficient quantity and be unexpired.

- **Separated workflows:** Order processing only consumes inventory; replenishment happens in dedicated checkpoint phases.
- **Baseline restocking:** Ingredients are replenished to predefined par levels.
- **Multi-reason tracking:** Ingredients can be flagged simultaneously for low stock and expiry with audit comments preserved.
- **Periodic replenishment:** Restocking occurs every N orders to balance operational recovery and system efficiency.

6. PROJECT_SPEC.md — Your AI's External Memory

AI has no persistent memory between sessions. Within a session, as the conversation grows, earlier instructions degrade or disappear entirely. To prevent context drift, every learner must maintain a PROJECT_SPEC.md file and paste the relevant section at the start of each new session.

What to Include in PROJECT_SPEC.md

- What you are building and why
- Key components and their current status
- Decisions already made (e.g. restock threshold = 1,000g, par level = 10,000g)
- Constraints and rules the AI must follow
- Current task and next task

How to Use It at the Start of Each Session

Session Re-anchor Prompt:

Here is our project context. We are building a Python-based cloud kitchen inventory simulation using seed_data.py (Recipes, Inventory, Orders, Restock, Status). We have completed Tasks 1-3. Task 4 (cumulative deduction) is the current task. Do not modify any previously implemented functions.

When to Update It

After every session: mark completed tasks, record new decisions, and add constraints that emerged. Treat it as the single source of truth for the project.

7. Prompt Templates Library

These four templates are reused throughout the entire project. Every prompt in Section 8 is built on one of these. Learn them once, apply them everywhere.

Use Templates 1 and 2 before writing a single line of code. They are your project architecture phase.

Use Template 3 strictly one task at a time — never bundle tasks.

Use Template 4 every time something breaks — never skip it and move on.

Template 1 Planning Prompt

When to use: At the very start of every new project or major component, before any code is written.

What it does: Gets AI thinking in components, not in code. Gives you a shared architecture to reference throughout.

Template:

"I want to build [PROJECT]. Before writing any code, give me a high-level plan — what components do I need and what should I build first?"

Template 2
Task Breakdown
Prompt

When to use: Run once per component, after the plan from Template 1 is confirmed.

What it does: Converts each component into individual, small, implementable tasks. Each task then becomes one Template 3 prompt. Small tasks = small errors = fast debugging.

Template:

"Here is component [NAME] from our plan. Break it into individual implementation tasks. Keep each task small enough to implement in a single prompt."

Template 3
Implementation
Prompt

When to use: For every individual task in the implementation phase. One task per prompt — never bundle tasks.

What it does: Keeps scope tight. The phrase "Do not implement any other tasks yet" prevents scope creep. Comments make the output readable and verifiable.

Template:

"Implement Task [N]: [DESCRIPTION]. Use Python. Keep the code simple and add comments explaining each section. Do not implement any other tasks yet."

Template 4
Debugging
Prompt

When to use: Every time a test fails or code produces unexpected output. Never skip it and move on.

What it does: Forces AI to explain the root cause, not just patch the error. This builds your understanding of the code over time so you genuinely own it.

Template:

"Here is the code and the error it produces: [PASTE CODE] / [PASTE ERROR]. Explain what is wrong and provide a corrected version with comments."

Correct Order of Use in Every Project

5. Template 1 — once, at the very start, before any code.
6. Template 2 — once per component, after the plan is confirmed.
7. Template 3 — for every individual task, one at a time.

8. Template 4 — whenever a test fails or an error appears.

Save these 4 templates. They cover the full development cycle: plan → breakdown → implementation → debugging.

8. Step-by-Step Instructions and Prompts

We follow a structured path divided into three parts: establish data connectivity first, define the operational flow second, and refine the logic to handle complex business rules last. Each step follows the Build → Test → Inspect → Refine loop from the course.

Part 1: Creating the Orchestrator

The goal is to ensure the application can “see” all the data it needs to work with. This layer acts as the foundation for the entire system.

Step 1: Establishing Data Connectivity

Goal: Create a baseline main.py that interfaces with every data structure in seed_data.py.

Note: seed_data.py contains 5 tables — Recipes (5 items: Margherita Pizza, Chicken Burger, Caesar Salad, Pasta Alfredo, Chocolate Cake with ingredient quantities in grams), Inventory (14 ingredients with current stock and expiry dates), Orders (5 orders across 2 brands — Taco Bell and Subway), Restock (5 flagged ingredients with reason), and Status (delivery status per order).

Planning Prompt (Template 1):

I want to build a Python cloud kitchen inventory simulation using seed_data.py. Before writing any code, give me a high-level plan — what components do I need and what should I build first?

Implementation Prompt (Template 3):

Implement Task 1: Create a baseline main.py file that imports all data from seed_data.py. Write specific functions to load and print the contents of every data table—including Recipes, Inventory, Orders, Restock, and Status—to the console. Add comments explaining each function. Do not implement any other tasks yet.

Validation Hook Prompt: After generating the code, add inline comments that identify: (1) any assumptions you made that I should verify, (2) any parts you are uncertain about, and (3) any sections that are incomplete or require follow-up.

Test Generation Prompt (immediately after Step 1 code): Write unit tests for the load functions using Python unittest. Cover: successful import of all 5 tables, correct count of records in each table, and correct data types for key fields (e.g. qty_grams is a number, expiry_date is a string).

5-Point Code Review Checklist — Apply After Every Step

Run this checklist on every function the AI generates:

9. Does it do what I asked? Run it with 3 different inputs you can verify manually.
10. Are there hidden assumptions? Look for hard-coded values, assumed data types, or logic paths you didn't specify.
11. Is anything hard-coded that should be a variable? (e.g. restock thresholds, par levels, expiry windows).
12. Does it handle bad inputs? What happens with None values, empty lists, or negative quantities?
13. Is any part of it security-sensitive or business-critical? (e.g. financial deductions, expiry logic).

Part 2: Defining the Application Flow

Now that the data is connected, we define the “rhythm” of the kitchen: taking an order, checking ingredients, and updating the status.

Step 2: Implementing Order-to-Recipe Processing

Goal: Create the logic to pull an order, look up its recipe, and identify required ingredients.

Task Breakdown Prompt (Template 2):

Here is component 'Order Processing' from our plan. Break it into individual implementation tasks. Keep each task small enough to implement in a single prompt.

Implementation Prompt (Template 3):

Implement Task 2: Update main.py to process orders from the Orders table. For each order, the system should: 1. Look up the recipe for each item. 2. Calculate the total grams of ingredients required. 3. Check the Inventory table to see if all required ingredients are present in the necessary quantities. Add comments explaining each step. Do not implement any other tasks yet.

Validation Hook Prompt: After generating the code, add inline comments that identify: (1) any assumptions you made about how recipe lookup works, (2) what happens if an order item has no matching recipe, and (3) any sections that are incomplete.

Test Generation Prompt: Write unit tests for the order-to-recipe lookup function using Python unittest. Cover: a valid order item with a matching recipe, an order item with no matching recipe (should handle gracefully), and correct calculation of total grams for an order with quantity 2.

Step 3: Fulfillment and Inventory Deduction

Goal: Update the order status and reduce stock if ingredients are available.

Implementation Prompt (Template 3):

Implement Task 3: Add fulfillment logic. If all ingredients are available, mark the order as 'Delivered' in the status table and deduct the used grams from the inventory. If ingredients are missing, mark the order as 'Not Delivered,' record the reason, and add the missing items to the Restock table. Add comments. Do not implement any other tasks yet.

Validation Hook Prompt: After generating the code, add inline comments identifying: (1) what happens if the same ingredient is missing from two consecutive orders, (2) any assumption about whether partial stock can be deducted, and (3) any incomplete sections.

Test Generation Prompt: Write unit tests for the fulfillment function. Cover: an order where all ingredients are available (expect Delivered), an order where one ingredient is missing (expect Not Delivered, missing item logged to Restock), and correct deduction from inventory after delivery.

Part 3: Refining the Logic for Correct Output

The final part ensures the inventory math is accurate across multiple orders and that restocking triggers correctly based on health rules.

Step 4: Cumulative Inventory Deduction

Goal: Ensure each order subtracts from the remaining inventory of the previous order.

Implementation Prompt (Template 3):

Implement Task 4: Refine the inventory logic to be cumulative. Order 2 must check availability against the quantity remaining after Order 1 was served. Update the final inventory table only after all orders in the list have been processed to show the true remaining stock. Add comments. Do not implement any other tasks yet.

Validation Hook Prompt: After generating the code, add inline comments identifying: (1) whether a deep copy or reference is used for the working inventory state, (2) what happens if two orders compete for the same ingredient, and (3) any incomplete sections.

Test Generation Prompt: Write unit tests for cumulative deduction. Cover: two orders consuming the same ingredient (verify combined deduction), an order that fails because a prior order consumed the remaining stock, and final inventory state matching expected remaining quantities.

Step 5: Implementing Restock and Expiry Rules

Goal: Automate restock reasons based on expiry dates and low-stock thresholds.

Implementation Prompt (Template 3):

Implement Task 5: Update the restock logic with these specific rules: 1. If an ingredient is within 5 days of expiry (relative to today's date), mark it as 'Expiring soon' and set the restock quantity to 10,000g. 2. If the final stock is 0, mark as 'Out of stock' (10,000g needed). 3. If stock is $\leq 1,000g$, mark as 'Running low on stock' and calculate the needed amount to reach 10,000g. Add comments. Do not implement anything beyond this task.

Validation Hook Prompt: After generating the code, add inline comments identifying: (1) how today's date is obtained — verify this is correct for the simulation, (2) whether an ingredient can be flagged for both low stock and expiry simultaneously, and (3) any incomplete sections.

Test Generation Prompt: Write unit tests for the restock logic. Cover: an ingredient expiring within 5 days (expect 'Expiring soon', 10,000g), an ingredient with 0 stock (expect 'Out of stock', 10,000g), an ingredient with 500g remaining (expect 'Running low', 9,500g needed), and an ingredient with adequate stock and no expiry issue (expect not flagged).

Duplication Check — After Every 3–4 Tasks

After completing Steps 2–5, run the following refactoring prompt before proceeding to enhancements. AI generates code based on each prompt in isolation and has no global view of your codebase — duplicate logic will accumulate silently.

Refactoring Prompt:

Review the code in main.py for duplicated logic or functions — particularly any inventory lookup, availability check, or restock flagging logic that appears more than once. Suggest how to refactor it to eliminate duplication while keeping the behaviour identical. Do not change any function signatures yet.

Debugging Loop — Template 4

When any test fails or any step produces unexpected output, do not skip ahead. Use this structured debugging loop.

- Step 1: Read the error message carefully — before anything else.
- Step 2: Try to understand what it is saying — what line, what type of error, what variable?
- Step 3: Use Template 4 — paste both code AND full error.
- Step 4: Ask AI to explain the root cause, not just give a fix.
- Step 5: Apply the fix and re-run ALL tests — not just the failing one.
- Step 6: If still failing, paste the new error and repeat.

Debugging Prompt (Template 4):

Here is the code and the error it produces: [PASTE FUNCTION CODE] / [PASTE FULL ERROR MESSAGE]. Explain what is wrong and why. Then provide a corrected version with comments explaining what was changed and why.

9. Session Exercise: Application Enhancements

Apply the full 8-step playbook independently for each enhancement. Do not skip the planning stage or jump straight to code. Each enhancement should follow: Plan → Task Breakdown → Implement one task → Validate hooks → Test → Debug → Refactor.

Enhancement 1: Partial Fulfillment Logic

Modify the order processing logic so that if a subset of items in an order can be fulfilled with remaining inventory, those items are delivered and marked separately rather than failing the entire order.

Starting Prompt (Template 1):

I want to add partial fulfillment logic to the cloud kitchen simulation. Before writing any code, give me a plan – what components need to change and what should I build first?

Reminder: After each implementation prompt, request validation hooks. After each function, request unit tests. Use Template 4 when tests fail.

Enhancement 2: Predictive Stockout Alerts

Enhance the system to calculate the current per-order consumption rate and forecast which ingredients will run out within the next batch of orders, even if they are currently in stock.

Starting Prompt (Template 1):

I want to add predictive stockout alerts to the cloud kitchen simulation. Before writing any code, give me a plan – what components do I need and what should I build first?

Enhancement 3: Dynamic Item Disabling

Implement a function that automatically identifies all menu items across all brands linked to a depleted ingredient and marks them as “unavailable” to prevent future order rejections.

Starting Prompt (Template 1):

I want to implement dynamic item disabling in the cloud kitchen simulation. Before writing any code, give me a plan – what components do I need and what should I build first?

10. Practical Approach Followed

- Import and print all data structures (Recipes, Inventory, Orders, Restock, Status) to establish full data connectivity.
- For each order, look up its recipe and calculate the total grams of ingredients required.
- Check inventory for availability; mark as Delivered and deduct stock, or mark as Not Delivered, log the reason, and add missing items to Restock.
- Make inventory deduction cumulative — each order checks against stock remaining after the previous order.
- Update the final inventory table only after all orders are processed.
- Apply restock rules to final inventory: Expiring soon (within 5 days, 10,000g); Out of stock (qty = 0, 10,000g); Running low (qty $\leq$ 1,000g, top-up to 10,000g).
- Generate unit tests immediately after each function is implemented.
- Use Template 4 (debugging prompt) whenever tests fail — always ask for root cause explanation, not just a fix.
- Request validation hooks after every implementation prompt to surface hidden assumptions.
- Run a duplication check every 3–4 tasks to prevent duplicate logic accumulating across steps.
- Maintain PROJECT_SPEC.md and re-anchor at the start of each new session.
- Refine outputs to a business-friendly reporting format.

11. Key Takeaways

01 The Business Pain Point

Inventory risk and delivery reliability are the real problems. Every technical decision should trace back to reducing stockouts, waste, or rejected orders.

02 The Technical Problem

The order-to-inventory-to-restock pipeline must be cumulative, expiry-aware, and threshold-triggered. Each component is a single, testable task.

03 The Implementation Strategy

Incremental AI prompts that evolve logic safely — with tests after every function, validation hooks after every prompt, a debugging loop when things break, and a context file to prevent drift.

This is not just writing code with AI. It is structured problem-solving with AI aligned to business outcomes.